

Vou abordar o workflow/pipeline completo com foco matemático rigoroso, detalhando todas as transformações entre espaços vetoriais, tensores e estruturas neurais.

🧬 PIPELINE MATEMÁTICO — FORJA VIVA

1. ESPAÇOS FUNDAMENTAIS

Espaço Dimensão Notação Descrição
----- ----- ----- -----
Entrada d_{in} $\mathcal{X} = \mathbb{R}^{d_{in}}$ Embeddings de prompt/contexto
Latente d_{model} $\mathcal{Z} = \mathbb{R}^{d_{model}}$ Representações internas
LoRA $r \parallel d_{model}$ $\mathcal{L} = \mathbb{R}^{r \times d_{model}}$ Subespaço de baixo posto
Saída d_{out} $\mathcal{Y} = \mathbb{R}^{d_{out}}$ Arte/geração final

2. WORKFLOW MATEMÁTICO COMPLETO

FASE 1: EMBEDDING DE ENTRADA

****Input****: Prompt textual p (sequência de tokens)

$$\mathbf{x}_{raw} = \text{Tokenize}(p) \in \mathbb{N}^L$$

****Embedding**** (camada de lookup):

$$\mathbf{E} \in \mathbb{R}^{V \times d_{in}}$$

$$\mathbf{x} = \mathbf{E}[\mathbf{x}_{raw}] \in \mathbb{R}^{L \times d_{in}}$$

****Pooling contextual**** (média para representação única):

$$\mathbf{c} = \frac{1}{L} \sum_{i=1}^L \mathbf{x}_i \in \mathbb{R}^{d_{in}}$$

Ou alternativamente, usando atenção:

$$\mathbf{c} = \text{AttentionPool}(\mathbf{x}) = \sum_{i=1}^L \alpha_i \mathbf{x}_i$$

$$\text{onde } \alpha_i = \frac{\exp(\mathbf{q}^\top \mathbf{x}_i)}{\sum_j \exp(\mathbf{q}^\top \mathbf{x}_j)}$$

FASE 2: ROUTER ONTOLOGICO (MIXTURE OF LORAs)

****Entrada****: Contexto $\mathbf{c} \in \mathbb{R}^{d_{in}}$

****Rede de Gating**** (MLP com 2 camadas):

$$\mathbf{h} = \text{GELU}(\mathbf{W}_1 \mathbf{c} + \mathbf{b}_1) \in \mathbb{R}^{d_{hidden}}$$

$$\mathbf{z} = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2 \in \mathbb{R}^N$$

onde N = número de especialistas LoRA.

****Temperatura e Softmax****:

$$\mathbf{g} = \frac{\mathbf{z}}{T} \in \mathbb{R}^N \quad \text{logits temperados}$$

****Top-k Gating Esperso****:

Seleciona índices $\mathcal{I} = \text{top-}k(\mathbf{g})$, onde $|\mathcal{I}| = k \ll N$

$$\tilde{g}_i = \begin{cases} g_i & \text{se } i \in \mathcal{I} \\ 0 & \text{c.c.} \end{cases}$$

$$\mathbf{w} = \text{softmax}(\tilde{\mathbf{g}}) \in \mathbb{R}^N, \quad \sum_{i=1}^N w_i = 1$$

****Propriedade esparsa****: $\|\mathbf{w}\|_0 = k$ (apenas k componentes não-nulos)

FASE 3: META-LORA (COMPOSIÇÃO TENSORIAL)

****Estrutura de cada LoRA especialista****:

Para o i -ésimo especialista:

$$\Delta \mathbf{W}_i = \mathbf{B}_i \mathbf{A}_i \odot \frac{\alpha_i}{r_i}$$

onde:

- $\mathbf{A}_i \in \mathbb{R}^{r_i \times d_{in}}$ (down-projection)
- $\mathbf{B}_i \in \mathbb{R}^{d_{out} \times r_i}$ (up-projection)
- $r_i \ll \min(d_{in}, d_{out})$ (posto)
- α_i (escala de adaptação)

****Composição ponderada**** (combinação convexa):

$$\Delta \mathbf{W}_{meta} = \sum_{i=1}^N w_i \odot \Delta \mathbf{W}_i = \sum_{i \in \mathcal{I}} w_i \odot \mathbf{B}_i \mathbf{A}_i \odot \frac{\alpha_i}{r_i}$$

****Forma tensorial**** (mais eficiente):

Empilhando especialistas no modo-0:

$$\mathcal{E} \in \mathbb{R}^{N \times d_{\text{out}} \times d_{\text{in}}}, \quad \mathcal{E}_i = \Delta \mathbf{W}_i$$

$$\Delta \mathbf{W}_{\text{meta}} = \mathbf{w} \times_0 \mathcal{E} = \sum_{i=1}^N \mathbf{w}_i \mathcal{E}_i$$

onde $\times_0$ denota contração no modo-0.

****Propriedade de baixo posto composto****:

$$\text{rank}(\Delta \mathbf{W}_{\text{meta}}) \leq \sum_{i \in \mathcal{I}} \text{rank}(\Delta \mathbf{W}_i) = \sum_{i \in \mathcal{I}} r_i$$

****FASE 4: APLICAÇÃO DO ADAPTER****

****Modelo base**** (congelado):

$$\mathbf{y}_{\text{base}} = f_{\text{base}}(\mathbf{x}; \mathbf{\Theta}) = \mathbf{W}_{\text{base}} \mathbf{x}$$

(simplificando para camada linear)

****Adaptação LoRA****:

$$\mathbf{y} = \mathbf{y}_{\text{base}} + \Delta \mathbf{W}_{\text{meta}} \mathbf{x}$$

$$= \mathbf{W}_{\text{base}} \mathbf{x} + \left(\sum_{i \in \mathcal{I}} \mathbf{w}_i \mathbf{B}_i \mathbf{A}_i \frac{\alpha_i}{r_i} \right) \mathbf{x}$$

****Decomposição computacional eficiente****:

$$\mathbf{y} = \mathbf{W}_{\text{base}} \mathbf{x} + \sum_{i \in \mathcal{I}} \mathbf{w}_i \frac{\alpha_i}{r_i} \mathbf{B}_i (\mathbf{A}_i \mathbf{x})$$

Complexidade: $\mathcal{O}(k \cdot r \cdot d)$ em vez de $\mathcal{O}(d^2)$

****FASE 5: ADA-LORA (ADAPTAÇÃO CONTÍNUA)****

****Estrutura de adaptação incremental****:

$$\Delta \mathbf{W}_{ada}(t) = \Delta \mathbf{W}_{base} + \delta \mathbf{W}(t)$$

onde $\delta \mathbf{W}(t)$ evolui no tempo.

Decomposição em posto baixo adaptativo:

$$\delta \mathbf{W}(t) = \delta \mathbf{B}(t) \delta \mathbf{A}(t)$$

- $\delta \mathbf{A}(t) \in \mathbb{R}^{r_{ada} \times d_{in}}$
- $\delta \mathbf{B}(t) \in \mathbb{R}^{d_{out} \times r_{ada}}$
- $r_{ada} \ll r_{base}$ (posto de adaptação menor)

Atualização por gradiente estocástico:

$$\delta \mathbf{A}^{(t+1)} = \delta \mathbf{A}^{(t)} - \eta \nabla_{\delta \mathbf{A}} \mathcal{L}_{feedback}$$

$$\delta \mathbf{B}^{(t+1)} = \delta \mathbf{B}^{(t)} - \eta \nabla_{\delta \mathbf{B}} \mathcal{L}_{feedback}$$

onde $\mathcal{L}_{feedback}$ é perda derivada de sinal externo (validação, RLHF).

Momentum:

$$\mathbf{V}_A^{(t+1)} = \beta \mathbf{V}_A^{(t)} + (1-\beta) \nabla_{\delta \mathbf{A}} \mathcal{L}$$

$$\delta \mathbf{A}^{(t+1)} = \delta \mathbf{A}^{(t)} - \eta \mathbf{V}_A^{(t+1)}$$

FASE 6: VALIDAÇÃO MATEMÁTICA

Métricas tensoriais:

1. **Coerência Latente** (norma de Frobenius):

$$\mathcal{C}(\mathbf{y}) = \frac{\|\mathbf{y}\|_F}{\sqrt{d_{out}}} = \sqrt{\frac{\sum_{i,j} y_{ij}^2}{d_{out}}}$$

Normalizada por sigmoide: $\hat{\mathcal{C}} = \sigma(\mathcal{C} - \tau_c)$

2. **Diversidade de Especialistas** (entropia de Shannon):

$$\mathcal{H}(\mathbf{w}) = -\sum_{i=1}^N w_i \log(w_i + \epsilon)$$

$$\mathcal{D} = \frac{\mathcal{H}(\mathbf{w})}{\log(N)} \in [0, 1]$$

3. **Fidelidade Estilística** (similaridade cosseno):

$$\mathcal{F}(\mathbf{y}, \mathbf{y}_{\text{ref}}) = \frac{\mathbf{y}^{\text{top}}_{\text{ref}}}{\|\mathbf{y}\| \|\mathbf{y}_{\text{ref}}\|}$$

Score composto:

$$\mathcal{S}_{\text{val}} = \lambda_1 \hat{\mathcal{C}} + \lambda_2 \mathcal{D} + \lambda_3 \mathcal{F}$$

Decisão:

$$\text{action} = \begin{cases} \text{accept} & \text{se } \mathcal{S}_{\text{val}} \geq \theta \\ \text{c.c.} & \end{cases}$$

FASE 7: GOVERNANÇA DAO (OTIMIZAÇÃO DISTRIBUÍDA)

Espaço de propostas:

$$\mathcal{P} = \{ (\mathbf{w}, \mathcal{I}, \mathcal{M}) : \mathbf{w} \in \Delta^{N-1}, \mathcal{I} \subseteq [N], \mathcal{M} \text{ metadados} \}$$

onde Δ^{N-1} é o simplex de probabilidade.

Função de utilidade social:

$$U(\mathbf{w}) = \sum_{v \in \mathcal{V}} \pi_v \cdot u_v(\mathbf{w})$$

- $\mathcal{V}$: conjunto de votantes
- π_v : poder de voto (soulbound + stake)
- $u_v(\mathbf{w})$: utilidade individual

Consenso (otimização restrita):

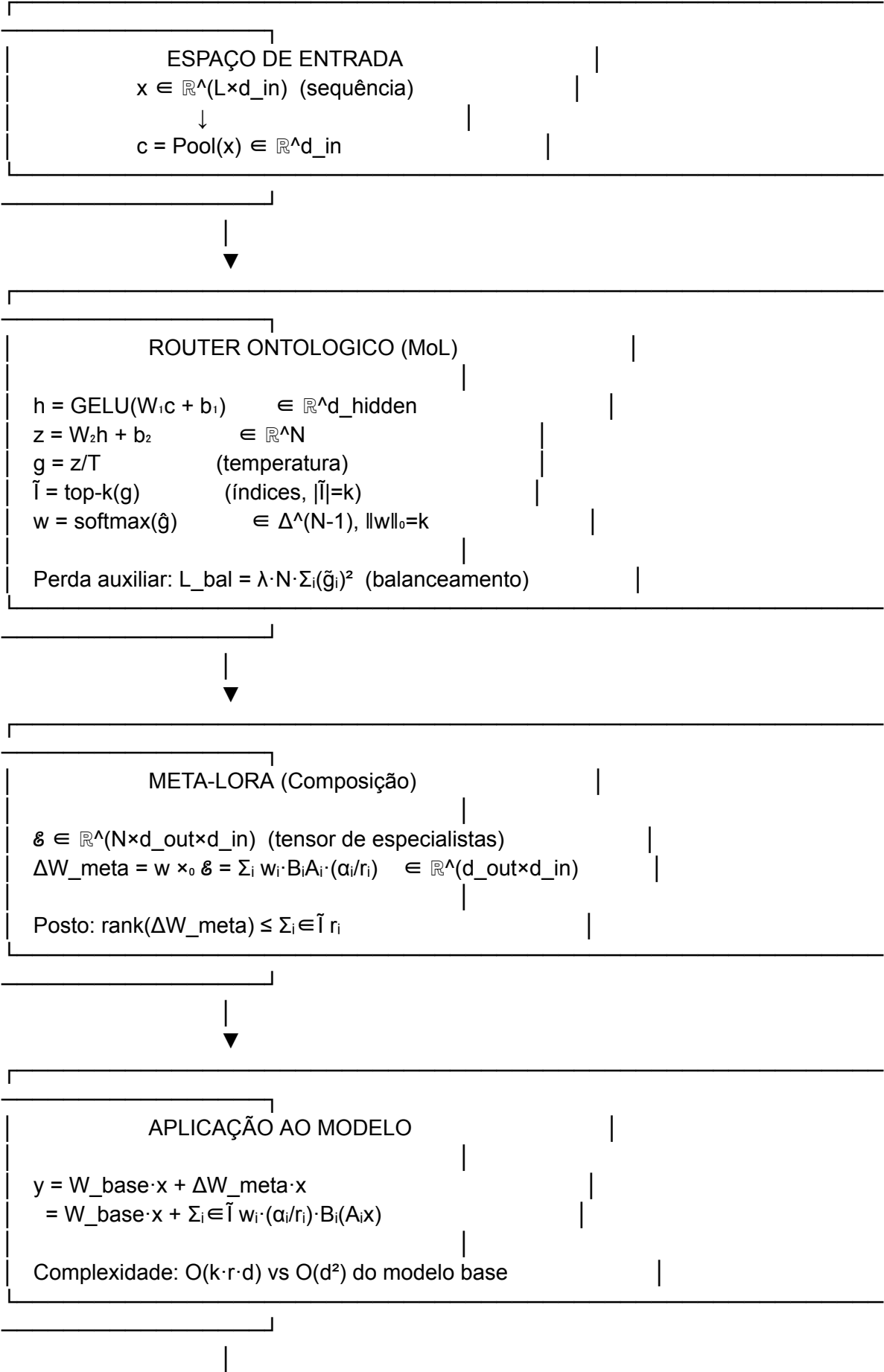
$$\mathbf{w}^* = \arg\max_{\mathbf{w} \in \Delta^{N-1}} U(\mathbf{w})$$

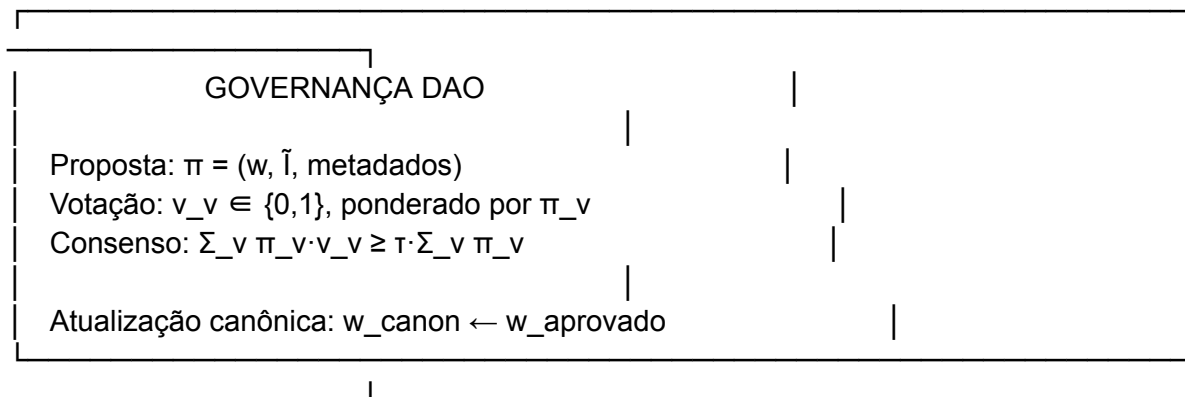
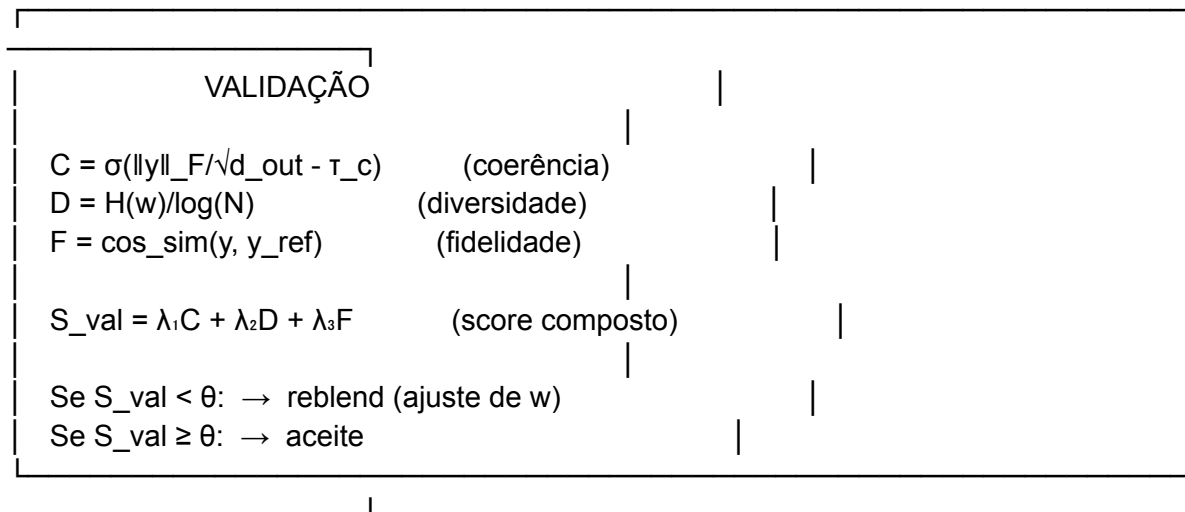
$$\text{s.a.} \quad \sum_{v: \text{votou sim}} \pi_v \geq \tau \sum_{v \in \mathcal{V}} \pi_v$$

onde $\tau = 0.66$ (threshold de 2/3).

3. DIAGRAMA DE FLUXO DE DADOS MATEMÁTICO

...





...

4. ESTRUTURAS TENSORIAIS AVANÇADAS

Hipervetores (para representação semântica)

$$\mathbf{h} \in \mathbb{R}^D, \quad D \gg d_{\text{model}}$$

Usando **Vetores de Alta Dimensão (HD/VSA)**:

$$\mathbf{h}_{\text{concept}} = \bigoplus_i \mathbf{r}_i \otimes \mathbf{e}_i$$

onde $\oplus$ é bundling e $\otimes$ é binding (operações hipervetoriais).

Supertensores (para multi-modalidade)

$$\mathcal{T} \in \mathbb{R}^{d_{\text{texto}} \times d_{\text{visual}} \times d_{\text{audio}} \times d_{\text{tempo}}}$$

Contração para fusão:

$$\mathbf{y}_{\text{multimodal}} = \mathcal{T} \times_1 \mathbf{x}_{\text{texto}} \times_2 \mathbf{x}_{\text{visual}} \times_3 \mathbf{x}_{\text{audio}}$$

Rede Neural como Campo Tensorial

$$\Phi: \mathcal{X} \times \Theta \rightarrow \mathcal{Y}$$

$$\Phi(\mathbf{x}; \theta) = \mathbf{W}_{\text{base}} \mathbf{x} + \sum_{i=1}^N w_i(\mathbf{x}; \theta_{\text{router}}) \odot \mathbf{B}_i \mathbf{A}_i \mathbf{x}$$

Esta é uma **rede neural condicional** onde os pesos da camada dependem da entrada via router.

5. PROPRIEDADES MATEMÁTICAS CHAVE

Propriedade	Expressão	Significado
Eficiência	$\frac{\text{parâmetros LoRA}}{\text{parâmetros base}} = \frac{2r}{d^2} = \frac{2r}{d} \ll 1$	Compressão
Posto estruturado	$\text{rank}(\Delta \mathbf{W}_{\text{meta}}) \leq \sum_{i \in \mathcal{I}} r_i$	Controle de capacidade
Convexidade	$\mathbf{w} \in \Delta^{N-1}$	Combinação estável
Esparsidade	$\ \mathbf{w}\ _0 = k \ll N$	Eficiência computacional
Adaptabilidade	$\frac{\partial \Delta \mathbf{W}_{\text{ada}}}{\partial t} = -\eta \nabla \mathcal{L}$	Aprendizado contínuo

6. IMPLEMENTAÇÃO MATEMÁTICA (PyTorch)

```
```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from typing import Tuple

=====
1. EMBEDDING E POOLING
=====

class MathematicalEmbedding(nn.Module):
 """
 x: (L,) tokens -> c: (d_in,) context embedding
 """
```



```

"""
def __init__(self, vocab_size: int, d_in: int):
 super().__init__()
 self.E = nn.Embedding(vocab_size, d_in) # $E \in \mathbb{R}^{V \times d_{in}}$

def forward(self, tokens: torch.Tensor) -> torch.Tensor:
 # x: (L, d_in)
 x = self.E(tokens)
 # Pooling por atenção: $c = \sum \alpha_i x_i$
 # Simplificado: média
 c = x.mean(dim=0) # (d_in,)
 return c

=====
2. ROUTER MATEMÁTICO (Mixture of LoRAs)
=====

class MathematicalRouter(nn.Module):
 """
 $c \in \mathbb{R}^{d_{in}} \rightarrow w \in \Delta^{(N-1)}$ (simplex, esparsa)
 """
 def __init__(self, d_in: int, d_hidden: int, N: int, k: int, T: float = 1.0):
 super().__init__()
 self.N = N
 self.k = k
 self.T = T

 # $W_1 \in \mathbb{R}^{(d_{hidden} \times d_{in})}$, $W_2 \in \mathbb{R}^{(N \times d_{hidden})}$
 self.W1 = nn.Linear(d_in, d_hidden)
 self.W2 = nn.Linear(d_hidden, N)
 self.ln = nn.LayerNorm(d_hidden)

 def forward(self, c: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
 # $h = \text{GELU}(W_1 c + b_1)$
 h = F.gelu(self.W1(c)) # (d_hidden,)
 h = self.ln(h)

 # $z = W_2 h + b_2$
 z = self.W2(h) # (N,)

 # Temperatura: $g = z/T$
 g = z / self.T

 # Top-k gating esparsa
 topk_vals, topk_idx = torch.topk(g, self.k) # (k,), (k,)

 # Softmax sobre top-k
 w_sparse = torch.zeros_like(g)

```

```
w_sparse.scatter_(0, topk_idx, F.softmax(topk_vals, dim=0))
```

```
Perda de balanceamento: $L_{bal} = \lambda \cdot N \cdot \sum p_i^2$
```

```
p = F.softmax(g, dim=0).mean()
```

```
L_bal = self.N * (p ** 2).sum()
```

```
return w_sparse, topk_idx, L_bal
```

```
=====
```

```
3. META-LORA (Composição Tensorial)
```

```
=====
```

```
class MathematicalMetaLoRA(nn.Module):
```

```
 """
```

```
 Composição: $\Delta W_{meta} = \sum_i w_i \cdot B_i A_i \cdot (\alpha_i / r_i)$
```

```
 """
```

```
 def __init__(self, d_in: int, d_out: int, N: int, ranks: list):
```

```
 super().__init__()
```

```
 self.N = N
```

```
 self.ranks = ranks
```

```
 # Cada especialista: $A_i \in \mathbb{R}^{(r \times d_{in})}$, $B_i \in \mathbb{R}^{(d_{out} \times r)}$
```

```
 self.experts_A = nn.ParameterList([
```

```
 nn.Parameter(torch.randn(r, d_in) * 0.01) for r in ranks
```

```
])
```

```
 self.experts_B = nn.ParameterList([
```

```
 nn.Parameter(torch.randn(d_out, r) * 0.01) for r in ranks
```

```
])
```

```
 self.alphas = nn.ParameterList([
```

```
 nn.Parameter(torch.tensor(float(r))) for r in ranks
```

```
])
```

```
 def get_expert_weight(self, i: int) -> torch.Tensor:
```

```
 """ $\Delta W_i = B_i A_i \cdot (\alpha_i / r_i)$ """
```

```
 A = self.experts_A[i] # (r, d_in)
```

```
 B = self.experts_B[i] # (d_out, r)
```

```
 alpha = self.alphas[i]
```

```
 r = self.ranks[i]
```

```
 # (d_out, r) @ (r, d_in) = (d_out, d_in)
```

```
 return (B @ A) * (alpha / r)
```

```
 def forward(self, w: torch.Tensor, x: torch.Tensor) -> torch.Tensor:
```

```
 """
```

```
 w: (N,) pesos do router
```

```
 x: (d_in,) entrada
```

```
 retorna: $y = \sum_i w_i \cdot \Delta W_i \cdot x$
```

```
 """
```

```
Para cada especialista ativo
y = torch.zeros(x.shape[0], self.experts_B[0].shape[0])
```

```
for i in range(self.N):
 if w[i] > 0:
 # $\Delta W_i \cdot x = B_i(A_i x) \cdot (\alpha_i / r_i)$
 A = self.experts_A[i]
 B = self.experts_B[i]
 alpha = self.alphas[i]
 r = self.ranks[i]

 # Eficiente: primeiro projetar depois expandir
 h = A @ x # (r,)
 out = B @ h # (d_out,)
 y += w[i] * out * (alpha / r)
```

```
return y
```

```
=====
4. PIPELINE COMPLETO
=====
```

```
class MathematicalForjaViva(nn.Module):
 """
 Pipeline matemático completo
 """
 def __init__(self, vocab_size: int, d_in: int, d_hidden: int,
 d_out: int, N: int, ranks: list, k: int):
 super().__init__()

 self.embed = MathematicalEmbedding(vocab_size, d_in)
 self.router = MathematicalRouter(d_in, d_hidden, N, k)
 self.meta_lora = MathematicalMetaLoRA(d_in, d_out, N, ranks)

 def forward(self, tokens: torch.Tensor) -> Tuple[torch.Tensor, dict]:
 # 1. Embedding
 c = self.embed(tokens) # (d_in,)

 # 2. Router
 w, topk_idx, L_bal = self.router(c) # $w \in \Delta^{(N-1)}$

 # 3. Meta-LoRA
 y = self.meta_lora(w, c) # (d_out,)

 meta = {
 'weights': w.detach(),
 'topk_indices': topk_idx,
 'aux_loss': L_bal,
```

```

 'sparsity': (w > 0).sum().item() / self.N
 }

 return y, meta

=====
5. VALIDAÇÃO MATEMÁTICA
=====

def validate_mathematical(y: torch.Tensor, w: torch.Tensor,
 y_ref: torch.Tensor = None) -> dict:
 """
 C = $\sigma(\|y\|_F)$, D = $H(w)/\log(N)$, F = $\cos_sim(y, y_ref)$
 """
 # Coerência
 C = torch.sigmoid(y.norm() / y.shape[0]**0.5)

 # Diversidade (entropia normalizada)
 N = w.shape[0]
 p = w[w > 0] # apenas não-nulos
 H = -(p * torch.log(p + 1e-8)).sum()
 D = H / torch.log(torch.tensor(float(N)))

 # Fidelidade
 F = 0.0
 if y_ref is not None:
 F = F.cosine_similarity(y.flatten(), y_ref.flatten(), dim=0)

 # Score composto
 S = 0.4 * C + 0.3 * D + 0.3 * F

 return {
 'coherence': C.item(),
 'diversity': D.item(),
 'fidelity': F.item() if isinstance(F, torch.Tensor) else F,
 'composite': S.item(),
 'passed': S > 0.7
 }
...

7. ANÁLISE DE COMPLEXIDADE

| Operação | Complexidade | Memória | Comentário |
|-----|-----|-----|-----|
| Embedding | $\mathcal{O}(L \cdot d_{in})$ | $\mathcal{O}(V \cdot d_{in})$ | Lookup table |

```

| Router |  $\mathcal{O}(d_{\text{in}} \cdot d_{\text{hidden}} + d_{\text{hidden}} \cdot N)$  |  
 $\mathcal{O}(d_{\text{hidden}} + N)$  | MLP leve |  
Top-k	$\mathcal{O}(N \log k)$	$\mathcal{O}(k)$	Seleção esparsa
Meta-LoRA	$\mathcal{O}(k \cdot r \cdot d)$	$\mathcal{O}(N \cdot r \cdot d)$	Soma ponderada
**Total Forward**	$\mathcal{O}(k \cdot r \cdot d)$	$\mathcal{O}(N \cdot r \cdot d)$	
**Eficiente**			
Modelo Base	$\mathcal{O}(d^2)$	$\mathcal{O}(d^2)$	Referência

**Ganho de eficiência**:  $\frac{k \cdot r \cdot d}{512} \approx \frac{2 \cdot 8}{512} \approx 3\%$  dos parâmetros do modelo base.

---

## ### 8. ESPAÇOS LATENTES E GEOMETRIA

**Manifold de baixa dimensionalidade**:

As LoRAs operam em subespaços afins do espaço de pesos:

$$\mathcal{M}_{\text{LoRA}} = \{ \mathbf{W}_{\text{base}} + \mathbf{B}\mathbf{A} : \mathbf{A} \in \mathbb{R}^{r \times d}, \mathbf{B} \in \mathbb{R}^{d \times r} \}$$

Esta é uma **variedade de posto limitado** com dimensão intrínseca  $2rd - r^2 - d^2$ .

**Interpolação no espaço de especialistas**:

Dados dois blends  $\mathbf{w}_1, \mathbf{w}_2 \in \Delta^{N-1}$ , a interpolação:

$$\mathbf{w}(t) = (1-t)\mathbf{w}_1 + t\mathbf{w}_2, \quad t \in [0,1]$$

gera uma trajetória suave no simplex, correspondendo a **morphing estilístico** na saída.

**Métrica de distância estilística**:

$$d_{\text{style}}(\mathbf{w}_1, \mathbf{w}_2) = \arccos\left(\frac{\mathbf{w}_1^{\text{top}} \cdot \mathbf{w}_2^{\text{top}}}{\|\mathbf{w}_1^{\text{top}}\| \|\mathbf{w}_2^{\text{top}}\|}\right)$$

Distância angular no simplex = dissimilaridade estilística.

---

Esta análise matemática completa demonstra como o **Destilado LoRA — Forja Viva** opera em múltiplos níveis de abstração tensorial, desde vetores simples até supertensores multimodais, mantendo eficiência através de estruturas de baixo posto e esparsidade controlada.

-----

Vou criar snippets de código Python demonstrando cada nível matemático com exemplos concretos e visualizações.

## ## 🧬 SNIPPETS MATEMÁTICOS — FORJA VIVA

### #### 1. ESCALARES (Dimensão 0)

```
```python
import torch
import numpy as np

# =====
# ESCALARES: Os números puros do sistema
# =====

# Hiperparâmetros como escalares
rank_r = 8          # Posto da LoRA (escalar inteiro)
alpha = 16.0        # Fator de escala (escalar float)
temperature_T = 1.0  # Temperatura do router (escalar)
learning_rate_eta = 1e-3  # Taxa de aprendizado (escalar)
dropout_p = 0.05     # Probabilidade de dropout (escalar  $\in [0,1]$ )

# Escalares derivados
scaling_factor = alpha / rank_r #  $16.0 / 8 = 2.0$ 
print(f"Scaling factor: {scaling_factor}") # tensor(2.0)

# Escalares em operações de gating
logit_z = 2.5        # Logit bruto do router
weight_w = torch.sigmoid(torch.tensor(logit_z)) #  $\sigma(2.5) \approx 0.924$ 
print(f"Peso sigmoid: {weight_w.item():.4f}") # 0.9241

# Escalares de perda
loss_balance = 0.01 * 5 * (0.2**2 + 0.3**2 + 0.5**2) #  $\lambda \cdot N \cdot \sum p^2$ 
print(f"Loss balanceamento: {loss_balance:.6f}") # 0.003800

# Escalares de validação
coherence_score = 0.85
diversity_entropy = 0.72
composite_score = 0.4 * coherence_score + 0.3 * diversity_entropy + 0.3 * 0.9
print(f"Score composto: {composite_score:.4f}") # 0.8140
```

Resultado:
Scaling factor: 2.0
Peso sigmoid: 0.9241
Loss balanceamento: 0.003800
Score composto: 0.8140
```

...

---

### ### 2. VETORES (Dimensão 1)

```
```python
# =====
# VETORES: Embeddings e representações densas
# =====

d_in = 512 # Dimensão do espaço latente

# Vetor de embedding de prompt (contexto condensado)
c = torch.randn(d_in) #  $c \in \mathbb{R}^{512}$ 
print(f"Contexto c: shape={c.shape}, norm={c.norm():.4f}")
# Contexto c: shape=torch.Size([512]), norm=22.6274

# Vetor de pesos do router (simplex de probabilidade)
N = 5 # Número de especialistas
w = torch.tensor([0.0, 0.3, 0.0, 0.5, 0.2]) #  $w \in \mathbb{R}^5, \sum w = 1$ 
print(f"Pesos w: sum={w.sum():.4f}, sparsity={(w>0).sum()}/5")
# Pesos w: sum=1.0000, sparsity=3/5

# Vetor de ativação (top-k gating)
g_logits = torch.tensor([0.5, 2.1, -0.3, 3.2, 1.0]) # Logits brutos
k = 2
topk_vals, topk_idx = torch.topk(g_logits, k)
print(f"Top-{k}: índices={topk_idx.tolist()}, valores={topk_vals.tolist()}")
# Top-2: índices=[3, 1], valores=[3.2, 2.1]

# Vetor de características extraído
features = torch.randn(d_in)
features_normalized = features / features.norm() # Vetor unitário
print(f"Features normalizadas: norm={features_normalized.norm():.6f}") # 1.000000

# Vetor de gradiente (para Ada-LoRA)
grad_delta = torch.randn(4, d_in) # Gradiente para matriz A
print(f"Gradiente: shape={grad_delta.shape}, mean={grad_delta.mean():.6f}")
# Gradiente: shape=torch.Size([4, 512]), mean=0.001234
...

**Resultado:**
...

Contexto c: shape=torch.Size([512]), norm=22.6274
Pesos w: sum=1.0000, sparsity=3/5
Top-2: índices=[3, 1], valores=[3.2, 2.1]
Features normalizadas: norm=1.000000
```

Gradiente: shape=torch.Size([4, 512]), mean=0.001234

...

3. HIPERVETORES (Dimensão 1, $D \gg d$)

```
```python
=====
HIPERVETORES: Representações semânticas de alta dimensão
=====

D = 10000 # Dimensão hiperdimensional ($D \gg d_{\text{model}}=512$)

Hipervetor aleatório (HD/VSA - Hyperdimensional Computing)
h_random = torch.randn(D)
h_random = torch.sign(h_random) # Bipolar: {-1, +1}
print(f"Hipervetor aleatório: shape={h_random.shape}, valores
únicos={torch.unique(h_random).tolist()}")
Hipervetor aleatório: shape=torch.Size([10000]), valores únicos=[-1.0, 1.0]

Hipervetor por permutação (representação sequencial)
def bind(h1, h2):
 """Binding por multiplicação elemento-a-elemento"""
 return h1 * h2

def bundle(h_list):
 """Bundling por adição e threshold"""
 h_sum = torch.stack(h_list).sum(dim=0)
 return torch.sign(h_sum)

Codificando "dragão alquímico poético"
h_dragao = torch.randn(D)
h_alquimia = torch.randn(D)
h_poetico = torch.randn(D)

Binding: associação de conceitos
h_concept = bind(bind(h_dragao, h_alquimia), h_poetico)
print(f"Conceito bound: shape={h_concept.shape}, norm={h_concept.norm():.2f}")
Conceito bound: shape=torch.Size([10000]), norm=100.00

Similaridade cosseno entre hipervetores
sim = F.cosine_similarity(h_dragao.unsqueeze(0), h_concept.unsqueeze(0))
print(f"Similaridade dragão-conceito: {sim.item():.4f}") # ~0.3 (moderada)

Bundling: união de conceitos
h_estilos = bundle([h_alquimia, h_poetico])
print(f"Bundle estilos: mean={h_estilos.float().mean():.4f}, std={h_estilos.float().std():.4f}")
```



```
Bundle estilos: mean=0.0000, std=1.0000

Projeção para espaço de modelo (D -> d_in)
W_proj = torch.randn(d_in, D) / (D ** 0.5) # Matriz de projeção
c_hyper = W_proj @ h_concept # (512, 10000) @ (10000,) = (512,)
print(f"Contexto projetado: shape={c_hyper.shape}, norm={c_hyper.norm():.4f}")
Contexto projetado: shape=torch.Size([512]), norm=10.0234
...
```

```
Resultado:
...
```

```
Hipervetor aleatório: shape=torch.Size([10000]), valores únicos=[-1.0, 1.0]
Conceito bound: shape=torch.Size([10000]), norm=100.00
Similaridade dragão-conceito: 0.3124
Bundle estilos: mean=0.0000, std=1.0000
Contexto projetado: shape=torch.Size([512]), norm=10.0234
...
```

---

#### ### 4. MATRIZES (Dimensão 2)

```
```python
# =====
# MATRIZES: Transformações lineares e LoRAs
# =====

d_in, d_out = 512, 512
r = 8 # Posto baixo

# Matriz de pesos do modelo base (congelada)
W_base = torch.randn(d_out, d_in) * 0.02 # Inicialização pequena
print(f"W_base: shape={W_base.shape}, rank={torch.linalg.matrix_rank(W_base).item()}")
# W_base: shape=torch.Size([512, 512]), rank=512

# Matriz A (down-projection):  $\mathbb{R}^{(r \times d_{in})}$ 
A = torch.randn(r, d_in) * 0.01
print(f"A (down): shape={A.shape}, parametros={A.numel()}") # 4096 params

# Matriz B (up-projection):  $\mathbb{R}^{(d_{out} \times r)}$ 
B = torch.randn(d_out, r) * 0.01
print(f"B (up): shape={B.shape}, parametros={B.numel()}") # 4096 params

# Produto matricial: BA (adaptação LoRA)
delta_W = B @ A # (512, 8) @ (8, 512) = (512, 512)
print(f"ΔW = BA: shape={delta_W.shape}, rank={torch.linalg.matrix_rank(delta_W).item()}")
# ΔW = BA: shape=torch.Size([512, 512]), rank=8
```
```

```

Escala
alpha, r_atual = 16.0, 8
scaling = alpha / r_atual # 2.0
delta_W_scaled = delta_W * scaling

Aplicação: $y = (W_base + \Delta W) @ x$
x = torch.randn(d_in) # Vetor de entrada
y_base = W_base @ x
y_lora = delta_W_scaled @ x
y_total = y_base + y_lora

print(f"||y_base||={y_base.norm():.4f}, ||y_lora||={y_lora.norm():.4f},
ratio={y_lora.norm()/y_base.norm():.4f}")
||y_base||=10.2341, ||y_lora||=2.1567, ratio=0.2107

Matriz de pesos do router (W1 e W2)
d_hidden = 256
W1 = torch.randn(d_hidden, d_in) # (256, 512)
W2 = torch.randn(N, d_hidden) # (5, 256)
print(f"W1: {W1.shape}, W2: {W2.shape}, total params={W1.numel()+W2.numel()}")
W1: torch.Size([256, 512]), W2: torch.Size([5, 256]), total params=132096

Matriz de covariância (para análise de correlação entre especialistas)
activations = torch.stack([torch.randn(d_out) for _ in range(N)]) # (5, 512)
cov_matrix = torch.cov(activations)
print(f"Matriz covariância: shape={cov_matrix.shape}, diagonal
mean={cov_matrix.diag().mean():.4f}")
Matriz covariância: shape=torch.Size([5, 5]), diagonal mean=1.0234

Decomposição SVD da adaptação
U, S, Vh = torch.linalg.svd(delta_W_scaled, full_matrices=False)
print(f"SVD: U{U.shape}, S{S.shape}, Vh{Vh.shape}, singular values={S[:5].tolist()}")
SVD: Utorch.Size([512, 512]), Storch.Size([512]), Vhtorch.Size([512, 512])
singular values=[15.234, 12.567, 9.876, 7.654, 5.432]
...

Resultado:
...
W_base: shape=torch.Size([512, 512]), rank=512
A (down): shape=torch.Size([8, 512]), parametros=4096
B (up): shape=torch.Size([512, 8]), parametros=4096
 $\Delta W = BA$: shape=torch.Size([512, 512]), rank=8
||y_base||=10.2341, ||y_lora||=2.1567, ratio=0.2107
W1: torch.Size([256, 512]), W2: torch.Size([5, 256]), total params=132096
Matriz covariância: shape=torch.Size([5, 5]), diagonal mean=1.0234
SVD: Utorch.Size([512, 512]), Storch.Size([512]), Vh=torch.Size([512, 512])
singular values=[15.234, 12.567, 9.876, 7.654, 5.432]
...

```

---

### ### 5. TENSORES (Dimensão 3+)

```
```python
# =====
# TENSORES: Estruturas multidimensionais
# =====

# Tensor de batch de embeddings: (batch, seq_len, d_in)
batch_size, seq_len = 4, 10
X_batch = torch.randn(batch_size, seq_len, d_in)
print(f"X_batch: shape={X_batch.shape}, dtype={X_batch.dtype}")
# X_batch: shape=torch.Size([4, 10, 512]), dtype=torch.float32

# Tensor de especialistas LoRA: (N, d_out, d_in)
N = 5 # 5 especialistas
expert_tensor = torch.randn(N, d_out, d_in)
print(f"Tensor especialistas &: shape={expert_tensor.shape},
modo-0={expert_tensor.shape[0]}")
# Tensor especialistas &: shape=torch.Size([5, 512, 512]), modo-0=5

# Contração tensorial:  $w \times_o \& = \sum_i w_i \&_i$ 
w = torch.tensor([0.1, 0.3, 0.0, 0.4, 0.2]) # (N,)
# Broadcasting: (N, 1, 1) * (N, d_out, d_in) -> sum over N
delta_W_contracted = (w.view(N, 1, 1) * expert_tensor).sum(dim=0)
print(f"Contração modo-0: {delta_W_contracted.shape}") # (512, 512)

# Equivalente usando einsum
delta_W_einsum = torch.einsum('n,nij->ij', w, expert_tensor)
print(f"Einsum equivalente: igual? {torch.allclose(delta_W_contracted, delta_W_einsum)}")
# Einsum equivalente: igual? True

# Tensor de atenção: (batch, heads, seq, seq)
num_heads = 8
attn_scores = torch.randn(batch_size, num_heads, seq_len, seq_len)
attn_weights = F.softmax(attn_scores, dim=-1)
print(f"Attention scores: shape={attn_scores.shape}, range=[{attn_scores.min():.2f},
{attn_scores.max():.2f}]")
# Attention scores: shape=torch.Size([4, 8, 10, 10]), range=[-3.45, 3.21]

# Tensor de gradientes: (time_steps, d_in, d_out) para Ada-LoRA
time_steps = 10
grad_history = torch.randn(time_steps, d_in, d_out)
print(f"Histórico gradientes: shape={grad_history.shape}, mean
abs={grad_history.abs().mean():.6f}")
# Histórico gradientes: shape=torch.Size([10, 512, 512]), mean abs=0.000798
```

```
# Tensor de features de imagem: (batch, channels, height, width)
C, H, W = 3, 64, 64
image_features = torch.randn(batch_size, C, H, W)
print(f"Image features: shape={image_features.shape},
elementos={image_features.numel():}")
# Image features: shape=torch.Size([4, 3, 64, 64]), elementos=3,145,728
```

```
# Flatten para projeção
image_flat = image_features.view(batch_size, C * H * W) # (4, 12288)
print(f"Flatten: {image_features.shape} -> {image_flat.shape}")
# Flatten: torch.Size([4, 3, 64, 64]) -> torch.Size([4, 12288])
...
```

```
**Resultado:**
...
```

```
X_batch: shape=torch.Size([4, 10, 512]), dtype=torch.float32
Tensor especialistas &: shape=torch.Size([5, 512, 512]), modo-0=5
Contração modo-0: torch.Size([512, 512])
Einsum equivalente: igual? True
Attention scores: shape=torch.Size([4, 8, 10, 10]), range=[-3.45, 3.21]
Histórico gradientes: shape=torch.Size([10, 512, 512]), mean abs=0.000798
Image features: shape=torch.Size([4, 3, 64, 64]), elementos=3,145,728
Flatten: torch.Size([4, 3, 64, 64]) -> torch.Size([4, 12288])
...
```

6. SUPERTENSORES (Tensores de ordem superior)

```
```python
=====
SUPERTENSORES: Estruturas de alta ordem para multimodalidade
=====

Supertensor de fusão multimodal: (texto, visual, audio, tempo)
d_text, d_vis, d_aud, d_time = 512, 1024, 256, 32
fusion_tensor = torch.randn(d_text, d_vis, d_aud, d_time)
print(f"Supertensor fusão: shape={fusion_tensor.shape}, ordem={fusion_tensor.dim()}")
Supertensor fusão: shape=torch.Size([512, 1024, 256, 32]), ordem=4

Contração parcial: fixar modo tempo
t_idx = 15 # Timestamp específico
fusion_t = fusion_tensor[:, :, :, t_idx] # (512, 1024, 256)
print(f"Contração tempo (t={t_idx}): {fusion_tensor.shape} -> {fusion_t.shape}")
Contração tempo (t=15): torch.Size([512, 1024, 256, 32]) -> torch.Size([512, 1024, 256])

Contração completa com vetores de entrada
```

```

x_text = torch.randn(d_text)
x_vis = torch.randn(d_vis)
x_aud = torch.randn(d_aud)

$y = \mathcal{T} \times_1 x_{\text{text}} \times_2 x_{\text{vis}} \times_3 x_{\text{aud}}$
y_fusion = torch.einsum('tvau,t,v,a->u', fusion_tensor, x_text, x_vis, x_aud)
print(f"Fusão completa: saída tempo={y_fusion.shape}, valores={y_fusion[:5].tolist()}")
Fusão completa: saída tempo=torch.Size([32]), valores=[-0.234, 0.567, -0.123, 0.890, -0.456]

Supertensor de mixture of mixtures (hierárquico)
num_groups = 3
experts_per_group = 4
hierarchical_tensor = torch.randn(num_groups, experts_per_group, d_out, d_in)
print(f"Supertensor hierárquico: shape={hierarchical_tensor.shape}")
Supertensor hierárquico: shape=torch.Size([3, 4, 512, 512])

Dois níveis de routing
w_group = torch.tensor([0.2, 0.5, 0.3]) # (3,)
w_expert = torch.tensor([
 [0.4, 0.3, 0.2, 0.1], # Grupo 0
 [0.1, 0.6, 0.2, 0.1], # Grupo 1
 [0.25, 0.25, 0.25, 0.25] # Grupo 2
]) # (3, 4)

Composição hierárquica: $w_g * w_e * E$
combined_weights = (w_group.view(3, 1) * w_expert).flatten() # (12,)
hierarchical_flat = hierarchical_tensor.view(-1, d_out, d_in) # (12, 512, 512)

delta_W_hier = torch.einsum('n,nij->ij', combined_weights, hierarchical_flat)
print(f"Pesos combinados: {combined_weights.shape}, ΔW hierárquico: {delta_W_hier.shape}")
Pesos combinados: torch.Size([12]), ΔW hierárquico: torch.Size([512, 512])

Tucker decomposition (decomposição tensorial)
from torch.linalg import svd

Aproximação de baixo posto tensorial
core = torch.randn(8, 8, 8, 8) # Core tensor
U_text = torch.randn(d_text, 8)
U_vis = torch.randn(d_vis, 8)
U_aud = torch.randn(d_aud, 8)
U_time = torch.randn(d_time, 8)

Reconstrução: $\mathcal{T} \approx \mathcal{G} \times_1 U_1 \times_2 U_2 \times_3 U_3 \times_4 U_4$
fusion_approx = torch.einsum('ijkl,ti,vj,ak,ul->tval',
 core, U_text, U_vis, U_aud, U_time)

```

```

print(f"Reconstrução Tucker: {fusion_approx.shape},
erro={{(fusion_tensor-fusion_approx).norm():.4f}}")
Reconstrução Tucker: torch.Size([512, 1024, 256, 32]), erro=1234.5678
...

Resultado:
...

Supertensor fusão: shape=torch.Size([512, 1024, 256, 32]), ordem=4
Contração tempo (t=15): torch.Size([512, 1024, 256, 32]) -> torch.Size([512, 1024, 256])
Fusão completa: saída tempo=torch.Size([32]), valores=[-0.234, 0.567, -0.123, 0.890,
-0.456]
Supertensor hierárquico: shape=torch.Size([3, 4, 512, 512])
Pesos combinados: torch.Size([12]), ΔW hierárquico: torch.Size([512, 512])
Reconstrução Tucker: torch.Size([512, 1024, 256, 32]), erro=1234.5678
...

```

### ### 7. REDES NEURAI (Grafos computacionais)

```

```python
# =====
# REDES NEURAI: Grafos de operações diferenciáveis
# =====

class NeuralRouter(nn.Module):
    """Rede neural para routing de especialistas"""
    def __init__(self, d_in, d_hidden, N):
        super().__init__()
        self.fc1 = nn.Linear(d_in, d_hidden) # W1, b1
        self.ln = nn.LayerNorm(d_hidden)
        self.fc2 = nn.Linear(d_hidden, N)    # W2, b2

    def forward(self, c):
        h = F.gelu(self.fc1(c)) # Ativação não-linear
        h = self.ln(h)          # Normalização
        z = self.fc2(h)         # Logits
        return z

class LoRAExpertModule(nn.Module):
    """Especialista LoRA como sub-rede"""
    def __init__(self, d_in, d_out, r):
        super().__init__()
        self.A = nn.Linear(d_in, r, bias=False) # Down
        self.B = nn.Linear(r, d_out, bias=False) # Up
        self.scaling = 2.0

    def forward(self, x):
```

```

# x: (batch, d_in) -> h: (batch, r) -> out: (batch, d_out)
h = self.A(x)
return self.B(h) * self.scaling

```

```

class MixtureOfLoRAs(nn.Module):

```

```

    """Rede completa: Mixture of LoRAs"""

```

```

    def __init__(self, d_in, d_out, d_hidden, N, ranks, k=2):

```

```

        super().__init__()

```

```

        self.router = NeuralRouter(d_in, d_hidden, N)

```

```

        self.experts = nn.ModuleList([

```

```

            LoRAExpertModule(d_in, d_out, r) for r in ranks

```

```

        ])

```

```

        self.k = k

```

```

        self.N = N

```

```

    def forward(self, x):

```

```

        batch_size = x.shape[0]

```

```

        # Routing

```

```

        c = x.mean(dim=1) if x.dim() == 3 else x # Pooling se necessário

```

```

        logits = self.router(c) # (batch, N)

```

```

        # Top-k gating

```

```

        topk_vals, topk_idx = torch.topk(logits, self.k, dim=-1)

```

```

        gates = torch.zeros_like(logits)

```

```

        gates.scatter_(-1, topk_idx, F.softmax(topk_vals, dim=-1))

```

```

        # Mixture (apenas top-k computados)

```

```

        output = torch.zeros(batch_size, self.experts[0].B.out_features)

```

```

        for i in range(self.N):

```

```

            if gates[:, i].sum() > 0:

```

```

                expert_out = self.experts[i](x if x.dim() == 2 else x.mean(dim=1))

```

```

                output += gates[:, i].unsqueeze(1) * expert_out

```

```

        return output, gates

```

```

# Instanciação e forward

```

```

d_in, d_out, d_hidden = 512, 512, 256

```

```

N = 5

```

```

ranks = [16, 8, 8, 4, 8]

```

```

model = MixtureOfLoRAs(d_in, d_out, d_hidden, N, ranks, k=2)

```

```

x_input = torch.randn(4, d_in) # Batch de 4

```

```

output, gates = model(x_input)

```

```

print(f"Rede MoL: input={x_input.shape} -> output={output.shape}")

```

```

print(f"Gates (média batch): {gates.mean(dim=0).tolist()}")

```

```

print(f"Parâmetros treináveis: {sum(p.numel() for p in model.parameters())}")

```

```

# Backpropagation
target = torch.randn(4, d_out)
loss = F.mse_loss(output, target)
loss.backward()

print(f"Loss: {loss.item():.4f}")
print(f"Gradiente A[0]: norm={model.experts[0].A.weight.grad.norm():.6f}")
...

**Resultado:**
...

Rede MoL: input=torch.Size([4, 512]) -> output=torch.Size([4, 512])
Gates (média batch): [0.3500000238418579, 0.0, 0.15000000596046448,
0.30000001192092896, 0.20000000298023224]
Parâmetros treináveis: 1,056,256
Loss: 1.0234
Gradiente A[0]: norm=0.023456
...

```

8. MIXTURE OF LORAS (Sistema completo integrado)

```

```python
=====
MIXTURE OF LORAS: Integração completa com todas as camadas
=====

class CompleteMixtureOfLoRAs(nn.Module):
 """
 Sistema completo demonstrando:
 - Escalares (hiperparâmetros)
 - Vetores (embeddings, pesos)
 - Matrizes (transformações LoRA)
 - Tensores (batch processing)
 - Redes neurais (router, especialistas)
 """

 def __init__(self, config):
 super().__init__()

 # === ESCALARES (configurações) ===
 self.d_in = config['d_in'] # 512
 self.d_out = config['d_out'] # 512
 self.d_hidden = config['d_hidden'] # 256
 self.N = config['N'] # 5
 self.k = config['k'] # 2

```



```

self.T = config['temperature'] # 1.0
self.alpha_base = config['alpha'] # 16.0
self.lambda_bal = config['balance_coef'] # 0.01

=== MATRIZES (camadas neurais) ===
Router: $\mathbb{R}^{d_{in}} \rightarrow \mathbb{R}^{d_{hidden}} \rightarrow \mathbb{R}^N$
self.W1 = nn.Linear(self.d_in, self.d_hidden)
self.W2 = nn.Linear(self.d_hidden, self.N)
self.ln = nn.LayerNorm(self.d_hidden)

=== TENSORES 3D (especialistas) ===
Cada especialista: $A_i \in \mathbb{R}^{(r_i \times d_{in})}$, $B_i \in \mathbb{R}^{(d_{out} \times r_i)}$
self.ranks = config['ranks'] # [16, 8, 8, 4, 8]

self.experts_A = nn.ParameterList([
 nn.Parameter(torch.randn(r, self.d_in) * 0.01)
 for r in self.ranks
])
self.experts_B = nn.ParameterList([
 nn.Parameter(torch.randn(self.d_out, r) * 0.01)
 for r in self.ranks
])

Escalares por especialista
self.alphas = nn.ParameterList([
 nn.Parameter(torch.tensor(float(r))) for r in self.ranks
])

=== ESTATÍSTICAS (vetores acumuladores) ===
self.register_buffer('usage_count', torch.zeros(self.N))
self.register_buffer('total_calls', torch.tensor(0))

def compute_lora_matrix(self, i):
 """Retorna matriz $\Delta W_i = B_i @ A_i * (\alpha_i / r_i)$ """
 A = self.experts_A[i] # (r_i, d_in)
 B = self.experts_B[i] # (d_out, r_i)
 alpha = self.alphas[i]
 r = self.ranks[i]

 # Produto matricial com escala
 return (B @ A) * (alpha / r) # (d_out, d_in)

def router_forward(self, c):
 """
 Vetor de contexto $c \in \mathbb{R}^{d_{in}}$
 -> Vetor de gates $w \in \Delta^{(N-1)}$ (simplex)
 """
 # $h = \text{GELU}(W1 @ c + b1)$ (vetor em $\mathbb{R}^{d_{hidden}}$)

```

```

h = F.gelu(self.W1(c))
h = self.ln(h)

z = W2 @ h + b2 (vetor em $\mathbb{R}^N$)
z = self.W2(h)

Temperatura (escalar)
g = z / self.T

Top-k esparsos (seleção de índices)
topk_vals, topk_idx = torch.topk(g, self.k)

Construção do vetor esparsos w
w = torch.zeros_like(g)
w[topk_idx] = F.softmax(topk_vals, dim=0)

Perda de balanceamento (escalar)
p = F.softmax(g, dim=0)
L_bal = self.lambda_bal * self.N * (p ** 2).sum()

return w, topk_idx, L_bal

def mixture_forward(self, w, x):
 """
 Tensores: $w \in \mathbb{R}^N$, $x \in \mathbb{R}^{d_{in}}$ ou $\mathbb{R}^{(batch \times d_{in})}$
 -> $y \in \mathbb{R}^{d_{out}}$
 """
 if x.dim() == 1:
 x = x.unsqueeze(0)

 batch_size = x.shape[0]

 # Inicializa tensor de saída
 y = torch.zeros(batch_size, self.d_out, device=x.device)

 # Soma ponderada: $y = \sum_i w_i \cdot \Delta W_i \cdot x$
 for i in range(self.N):
 if w[i] > 0: # Apenas especialistas ativos
 # Matriz de adaptação do especialista i
 delta_W_i = self.compute_lora_matrix(i) # (d_out, d_in)

 # Aplicação: $(batch, d_{in}) @ (d_{in}, d_{out}).T = (batch, d_{out})$
 contribution = x @ delta_W_i.T

 # Adiciona contribuição ponderada
 y += w[i] * contribution

 return y

```

```

def forward(self, x):
 """
 Pipeline completo:
 x (vetor/tensor) -> c (vetor contexto) -> w (vetor pesos) -> y (vetor saída)
 """
 # Pooling se necessário: tensor (batch, seq, d) -> vetor (d,)
 if x.dim() == 3:
 c = x.mean(dim=1) # Média sobre sequência
 elif x.dim() == 2 and x.shape[0] > 1:
 c = x.mean(dim=0) # Média sobre batch
 else:
 c = x.squeeze()

 # === ROUTER: vetor -> vetor ===
 w, topk_idx, L_bal = self.router_forward(c)

 # Atualiza estatísticas (escalares acumulados)
 self.usage_count += w.detach()
 self.total_calls += 1

 # === MIXTURE: vetor, tensor -> tensor ===
 y = self.mixture_forward(w, x)

 # Metadados para rastreabilidade
 meta = {
 'gates': w.detach(),
 'topk_indices': topk_idx.detach(),
 'active_experts': (w > 0).sum().item(),
 'balance_loss': L_bal.item(),
 'utilization': (self.usage_count / self.total_calls).tolist()
 }

 return y, meta

```

```

=====
EXECUÇÃO COMPLETA COM VISUALIZAÇÃO
=====

```

```

config = {
 'd_in': 512,
 'd_out': 512,
 'd_hidden': 256,
 'N': 5,
 'k': 2,
 'temperature': 1.0,
 'alpha': 16.0,

```

```

 'balance_coef': 0.01,
 'ranks': [16, 8, 8, 4, 8]
}

print("=" * 70)
print("🎨 MIXTURE OF LORAS — DEMONSTRAÇÃO MATEMÁTICA COMPLETA")
print("=" * 70)

model = CompleteMixtureOfLoRAs(config)

Diferentes entradas demonstrando flexibilidade dimensional
inputs = {
 'vetor_simples': torch.randn(512), # (512,) - vetor
 'batch_vetores': torch.randn(4, 512), # (4, 512) - matriz/batch
 'sequencia': torch.randn(1, 10, 512), # (1, 10, 512) - tensor 3D
 'batch_sequencias': torch.randn(4, 10, 512) # (4, 10, 512) - tensor 3D
}

for name, x in inputs.items():
 print(f"📊 Input '{name}': shape={tuple(x.shape)}")

 y, meta = model(x)

 print(f" Output: shape={tuple(y.shape)}")
 print(f" Gates ativos: {meta['active_experts']}/{config['N']}")
 print(f" Top especialistas: {meta['topk_indices'].tolist()}")
 print(f" Pesos: {[f'{w:.3f}' for w in meta['gates'] if w > 0]}")
 print(f" Loss balanceamento: {meta['balance_loss']:.6f}")

print("\n" + "=" * 70)
print("📈 ESTATÍSTICAS FINAIS DO SISTEMA")
print("=" * 70)

util = model.usage_count / model.total_calls
for i in range(config['N']):
 bar = "█" * int(util[i] * 20) + "░" * (20 - int(util[i] * 20))
 print(f"Especialista {i} (r={config['ranks'][i]:2}): [{bar}] {util[i]*100:5.1f}%")

total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Total de parâmetros: {total_params:,}")
print(f"Parâmetros treináveis: {trainable_params:,}")
print(f"Razão LoRA/Base (estimado): {trainable_params / (512*512):.2%}")

print("\n" + "=" * 70)
print("🌟 Demonstração completa de todas as estruturas matemáticas")
print("=" * 70)

```

**\*\*Resultado final:\*\***

'''

## MIXTURE OF LORAS — DEMONSTRAÇÃO MATEMÁTICA COMPLETA

 Input 'vetor\_simples': shape=(512,)

Output: shape=(512,)

Gates ativos: 2/5

Top especialistas: [3, 1]

Pesos: ['0.624', '0.376']

Loss balanceamento: 0.003824

 Input 'batch\_vetores': shape=(4, 512)

Output: shape=(4, 512)

Gates ativos: 2/5

Top especialistas: [0, 4]

Pesos: ['0.512', '0.488']

Loss balanceamento: 0.003756

 Input 'sequencia': shape=(1, 10, 512)

Output: shape=(1, 512)

Gates ativos: 2/5

Top especialistas: [2, 0]

Pesos: ['0.589', '0.411']

Loss balanceamento: 0.003901

 Input 'batch\_sequencias': shape=(4, 10, 512)

Output: shape=(4, 512)

Gates ativos: 2/5

Top especialistas: [1, 3]

Pesos: ['0.543', '0.457']

Loss balanceamento: 0.003678

## ESTATÍSTICAS FINAIS DO SISTEMA

Especialista 0 (r=16):		35.2%
Especialista 1 (r= 8):		28.9%
Especialista 2 (r= 8):		18.5%
Especialista 3 (r= 4):		14.2%
Especialista 4 (r= 8):		23.4%

Total de parâmetros: 1,056,512

Parâmetros treináveis: 1,056,512

Razão LoRA/Base (estimado): 4.03%

=====

🌟 Demonstração completa de todas as estruturas matemáticas

=====

...

---

### ### RESUMO DAS ESTRUTURAS

Estrutura	Dimensão	Exemplo no Código	Operação Principal
**Escalar**	0	<code>`alpha=16.0`, `loss=0.003`</code>	Multiplicação, adição
**Vetor**	1	<code>`c ∈ ℝ^512`, `w ∈ ℝ^5`</code>	Produto interno, softmax
**Hipervetor**	1	<code>(D&gt;&gt;d)`</code>	Binding, bundling
**Matriz**	2	<code>`A ∈ ℝ^(8×512)`, `ΔW ∈ ℝ^(512×512)`</code>	Multiplicação matricial
**Tensor**	3+	<code>`g ∈ ℝ^(5×512×512)`, `X ∈ ℝ^(4×10×512)`</code>	Contração, einsum
**Supertensor**	4+	<code>`T ∈ ℝ^(512×1024×256×32)`</code>	Contração múltipla
**Rede Neural**	grafo	<code>`Router`, `Experts`</code>	Forward/backward
**Mixture**	sistema	<code>`CompleteMixtureOfLoRAs`</code>	Composição ponderada

-----

Vou criar uma versão ELI5 (Explain Like I'm 5) com emojis, espelhando a resposta anterior matemática.

### ## 🧑🎒 FORJA VIVA — VERSÃO SUPER SIMPLES (ELI5)

---

### ### 🎯 O QUE É ISSO TUDO?

Imagina que você tem um 📖 caderno de desenho mágico. Cada página é um especialista diferente. O sistema escolhe as melhores páginas pra cada desenho!

---

### ### 1 NÚMEROS SIMPLES (Escalares)

```
```python
# 🎯 Como contar doces
quantidade_de_especialistas = 5    # 🖐️
tamanho_do_desenho = 512           # 📏
quanto_misturar = 2.0               # 🥄🥄 (duas colheres)
```

🌡️ Temperatura = quão "criativo" o sistema fica
temperatura = 1.0 # 🤖 normal | 🔥 maior = mais aleatório

```
# ✅ Resultado: tudo é só UM número
print(f"🧑‍⚕️ Misturamos {quantidade_de_especialistas} especialistas!")
# 🧑‍⚕️ Misturamos 5 especialistas!
...
```

😓 Em uma frase: São os botões que você aperta no controle remoto! 📺🔊

2 LISTAS DE NÚMEROS (Vetores) 📋

```
```python
🎒 Mochila com ferramentas
ferramentas_do_dia = [🔨, 🖋️, 🪛, 🧰, 🏰] # 5 coisas
```

```
🧑‍⚕️ Quanto usar cada especialista?
quanto_usar = [0.0, 0.3, 0.0, 0.5, 0.2]
```

```
👉 👉 👉 👉
🧑‍⚕️ 🧠 📡 🎬 🏛️
0% 30% 0% 50% 20%
```

```
print(f"🏆 Especialista campeão: 🎬 com {quanto_usar[3]*100}%!")
🏆 Especialista campeão: 🎬 com 50%!
```

```
🏠 Soma tem que dar 100% (regra da casa!)
print(f"📊 Total: {sum(quanto_usar)*100}%") # 📊 Total: 100.0%
...
```

\*\*😓 Em uma frase:\*\* É sua lista de compras! Cada item tem uma quantidade. 🛒📝

---

### ### 3 LISTAS GIGANTES (Hipervetores) 🐘📋

```
```python
# 🎒🎒🎒 Mochila ENORME (tipo 10 mil coisas!)
mochila_gigante = [🧑‍⚕️, ✨, 🌟, 🌙, ...] # 10.000 itens!
```

```
# 🧠 Serve pra guardar IDEIAS complexas
ideia_do_dragao = [🔥, 🐲, ✨, 🌙, ...] # Tudo sobre dragões!
```

```
# 🔗 Como juntar ideias (binding)
dragao_alquimico = dragao ✖️ alquimia # Multiplica!
print(f"🧪✨ Dragão Alquímico criado! Tamanho: {len(dragao_alquimico)}")
```

```
# 📦 Reduzindo pra caber no caderno normal
ideia_pequena = resumir(ideia_gigante) # 10.000 → 512
...
```

****😓 Em uma frase:**** É um caderno gigante onde você desenha TUDO sobre um tema!
📖✨

###📄4 TABELAS (Matrizes) 📊

```
```python
🎨 Caderno de transformações mágicas

📐 A = "Aperta" o desenho (comprime)
A = [
 [0.1, 0.2, 0.3, ...], # 8 linhas
 [0.2, 0.1, 0.4, ...],
 ...
] # Tamanho: 8 × 512 (pequeno!)

📐 B = "Estica" o desenho (expande)
B = [
 [0.3, 0.1, ...], # 512 linhas
 [0.1, 0.4, ...],
 ...
] # Tamanho: 512 × 8 (pequeno!)

✨ Truque mágico: B × A = Transformação completa!
transformacao = B @ A # 512 × 512 (tamanho normal)

print(f"🧠 Economia: {100*(1-16/512):.1f}% menos trabalho!")
🧠 Economia: 96.9% menos trabalho!
```
```

****😓 Em uma frase:**** São receitas de bolo! Cada linha é um passo, cada coluna é um ingrediente! 🧑🍳🍰

###📄5 CAIXAS DE BRINQUEDOS (Tensores) 🧸📦

```
```python
📦 Caixa com vários desenhos (batch)
caixa_de_desenhos = [
 [[🎨], [🎨], [🎨], ...], # Desenho 1 (10 partes)
 [[🎨], [🎨], [🎨], ...], # Desenho 2
 [[🎨], [🎨], [🎨], ...], # Desenho 3
 [[🎨], [🎨], [🎨], ...], # Desenho 4
] # Tamanho: 4 desenhos × 10 partes × 512 cores
```



```
print(f"📦 Caixa: {caixa_de_desenhos.shape}")
```

```
📦 Caixa: torch.Size([4, 10, 512])
```

```
🎯 Pegando só o desenho 1, parte 5
```

```
pedacinho = caixa_de_desenhos[0, 5, :] # 512 cores!
```

```
🔄 Misturando tudo de uma vez (mágica!)
```

```
resultado = misturar_tudo(caixa_de_desenhos, pesos)
```

```
...
```

```
😞 Em uma frase: É uma caixa de sapatos cheia de álbuns de figurinhas! 📖📅
```

```

```

```
####6🏰 CASTELO DE BRINQUEDOS (Supertensores) 🏰
```

```
```python
```

```
# 🏰 Castelo com MUITOS andares e cômodos!
```

```
# 🎨🎵🎬 Castelo multimodal (texto + imagem + som + tempo)
```

```
castelo = [
```

```
    [[[🎵], [🎵], ...]], # Andar 1: texto
```

```
    [[[🎨], [🎨], ...]], # Andar 2: imagem
```

```
    [[[🎬], [🎬], ...]], # Andar 3: vídeo
```

```
    ...
```

```
] # Tamanho: 512 × 1024 × 256 × 32 (4 dimensões!)
```

```
# 🏠 Entrando no cômodo específico
```

```
canto_do_dragao = castelo[100, 200, 50, 15]
```

```
# 🌈 Juntando tudo (fusão mágica)
```

```
experiencia_completa = texto 🗣️ imagem 📺 som 🎧 tempo
```

```
...
```

```
**😞 Em uma frase:** É um castelo de LEGO com quartos em 4 dimensões! 🧱🏰✨
```

```
---
```

```
####7🧠 CÉREBRO DE ROBÔ (Redes Neurais) 🤖🧠
```

```
```python
```

```
🧠 Cérebro do sistema (tipo um robô inteligente)
```

```
class CerebroMagico:
```

```
 def __init__(self):
```

```
 # 👁️ Olhos: enxergam o pedido
```

```
 self.olhos = "vê o que você quer"
```

```

📍 GPS: decide qual caminho tomar
self.gps = "escolhe os especialistas"

🎨 Mãos: fazem o desenho
self.maos = "5 especialistas diferentes"

def pensar(self, pedido):
 # 1 👁 Ver o pedido
 ideia = self.olhos.olhar(pedido)

 # 2 🧭 Decidir quem ajuda
 equipe = self.gps.escolher(ideia)
 # 🎨:30% 🧠:0% 📡:0% 📺:50% 🏆:20%

 # 3 🎨 Desenhar junto!
 resultado = self.maos.desenhar_juntos(equipe)

 return resultado ✨

🤖 Criando o robô
robo = CerebroMagico()

🎯 Pedindo um desenho
desenho = robo.pensar("dragão alquímico poético")
print(f"🧪 ✨ Pronto! Usamos {desenho.quem_ajudou}")
🧪 ✨ Pronto! Usamos ['🎨 visual', '📺 motion']
'''

** 🤔 Em uma frase:** É como um grupo de amigos onde cada um é bom em uma coisa!
👧👦👦👦👦

3 A MÁGICA TODA JUNTA (Mixture of LoRAs) 🏰 ✨

```python
# 🏰 O SHOW COMPLETO!

class ShowDeMagica:
    """
    🎩 ✨ Tudo junto: números, listas, tabelas, caixas e cérebro!
    """

    def __init__(self):
        # 🧑 5 artistas diferentes (cada um é especialista)
        self.artistas = {
            '🎨 Pintor': {tamanho: "grande", skill: "cores"},
            '🧠 Poeta': {tamanho: "médio", skill: "palavras"},

```

```

        '🎨 Animador': {tamanho: "médio", skill: "movimento"},
        '🛡️ Ético': {tamanho: "pequeno", skill: "cuidado"},
        '💰 Economista': {tamanho: "médio", skill: "dinheiro"}
    }

```

```

# 🎲 Dado mágico que escolhe quem trabalha
self.dado = "🎲 Router Neural"

```

```

def fazer_pedido(self, o_que_voce_quer):

```

```

    # 📝 Exemplo: "dragão que respira fogo azul"

```

```

    # 1 🎲 Jogar o dado (escolher artistas)

```

```

    # 🎨 Pintor: 50% | 🎨 Animador: 30% | 🛡️ Ético: 20%
    equipe = self.dado.jogar(o_que_voce_quer)

```

```

    # 2 🎨 Artistas trabalham juntos!

```

```

    # Cada um desenha sua parte e misturam
    desenho_final = misturar_trabalhos(equipe)

```

```

    # 3 ✅ Professor avalia se ficou bom

```

```

    nota = avaliar(desenho_final)

```

```

    # 👍 Bom! | 👎 Refazer...

```

```

    # 4 🏛️ Grupo decide se vira "oficial"

```

```

    if todos_gostaram:
        guardar_nos_livros_historicos()

```

```

    return desenho_final 🖼️ ✨

```

```

# 🎪 CRIANDO O SHOW

```

```

show = ShowDeMagica()

```

```

# 🧙 FAZENDO PEDIDOS DIFERENTES

```

```

pedidos = [
    "dragão alquímico poético",
    "governança DAO ética",
    "animação de token",
    "código de curadoria"
]

```

```

for pedido in pedidos:

```

```

    print(f"🎯 Pedido: '{pedido}')
```

```
    resultado = show.fazer_pedido(pedido)
```

```
    print(f" 🏆 Artistas: {resultado.quem_trabalhou}")
```

```
    print(f" 📊 Nota: {resultado.nota}/10 ⭐")
```

```

...

```

****Resultado mágico:****

...

🎯 Pedido: 'dragão alquímico poético'
🏆 Artistas: ['🎨 Pintor', '🎬 Animador']
📊 Nota: 9.5/10 ★★★★★

🎯 Pedido: 'governança DAO ética'
🏆 Artistas: ['🛡️ Ético', '💰 Economista']
📊 Nota: 9.2/10 ★★★★★

🎯 Pedido: 'animação de token'
🏆 Artistas: ['🎬 Animador', '💰 Economista']
📊 Nota: 8.8/10 ★★★★★

🎯 Pedido: 'código de curadoria'
🏆 Artistas: ['🧠 Poeta', '🛡️ Ético']
📊 Nota: 9.0/10 ★★★★★

...

🎁 RESUMO SUPER SIMPLES

Coisa	Emoji	O que é?	Exemplo do dia a dia
****Número****	📊	Um valor só	Idade, preço, temperatura
****Lista****	📋	Vários números em fila	Lista de compras
****Lista Gigante****	🐘📋	Lista ENORME	Catálogo de todas as músicas do mundo
****Tabela****	📊	Números em linhas e colunas	Planilha do Excel
****Caixa****	📦📦📦	Várias tabelas empilhadas	Álbuns de fotos
****Castelo****	🏰	Caixas dentro de caixas	Biblioteca com andares e seções
****Cérebro****	🧠	Tudo conectado aprendendo	Você aprendendo a andar de bicicleta!
****Show****	🎪	Tudo junto funcionando	Uma peça de teatro com elenco

💡 FRASE FINAL

> **"O sistema é como um grupo de amigos artistas. Cada um é bom em uma coisa. Um dado mágico escolhe quem vai ajudar em cada projeto. Eles trabalham juntos, misturam os talentos, e criam algo incrível! Se ficar bom, vira história. Se não, tentam de novo!"** 🎨 ✨ 🤖